

13. Construct a C program to implement various memory allocation strategies.

AIM:

To construct a C program to implement various memory allocation strategies.

CODE:

```
#include <stdio.h>
```

```
void sort(int a[], int n)
{
    for (int i = 0; i < n; i++)
    {
        for (int j = i + 1; j < n; j++)
        {
            if (a[i] > a[j])
            {
                int t = a[i];
                a[i] = a[j];
                a[j] = t;
            }
        }
    }
}
```

```
void rsort(int a[], int n)
{
    for (int i = 0; i < n; i++)
    {
        for (int j = i + 1; j < n; j++)
```

```

    {
        if (a[i] < a[j])
        {
            int t = a[i];
            a[i] = a[j];
            a[j] = t;
        }
    }
}

```

```

void bestfit(int mp[], int p[], int n, int m)
{
    int allocated[20] = {0};

    for (int i = 0; i < m; i++)
    {
        int best = -1;

        for (int j = 0; j < n; j++)
        {
            if (!allocated[j] && mp[j] >= p[i])
            {
                if (best == -1 || mp[j] < mp[best])
                    best = j;
            }
        }
    }
}

```

```

    if (best != -1)
    {
        printf("\n%d fits in %d", p[i], mp[best]);
        mp[best] -= p[i];
    }
    else
    {
        printf("\n%d must wait for its process", p[i]);
    }
}
}

```

```

void firstfit(int mp[], int p[], int n, int m)
{
    for (int i = 0; i < m; i++)
    {
        int found = 0;

        for (int j = 0; j < n; j++)
        {
            if (mp[j] >= p[i])
            {
                printf("\n%d fits in %d", p[i], mp[j]);
                mp[j] -= p[i];
                found = 1;
                break;
            }
        }
    }
}

```

```

    }
}

if (!found)
    printf("\n%d must wait for its process", p[i]);
}
}

```

```

void worstfit(int mp[], int p[], int n, int m)
{
    for (int i = 0; i < m; i++)
    {
        int worst = -1;

        for (int j = 0; j < n; j++)
        {
            if (mp[j] >= p[i])
            {
                if (worst == -1 || mp[j] > mp[worst])
                    worst = j;
            }
        }

        if (worst != -1)
        {
            printf("\n%d fits in %d", p[i], mp[worst]);
            mp[worst] -= p[i];
        }
    }
}

```

```

    }
    else
    {
        printf("\n%d must wait for its process", p[i]);
    }
}
}

```

```

int main()
{
    int m, n, mp[20], p[20], ch;

    printf("Number of memory partitions: ");
    scanf("%d", &n);

    printf("Number of processes: ");
    scanf("%d", &m);

    printf("Enter the memory partitions:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &mp[i]);

    printf("Enter process sizes:\n");
    for (int i = 0; i < m; i++)
        scanf("%d", &p[i]);

    printf("\n1. First Fit");
}

```

```
printf("\n2. Best Fit");
printf("\n3. Worst Fit");
printf("\nEnter your choice: ");
scanf("%d", &ch);

switch (ch)
{
    case 1:
        firstfit(mp, p, n, m);
        break;

    case 2:
        bestfit(mp, p, n, m);
        break;

    case 3:
        worstfit(mp, p, n, m);
        break;

    default:
        printf("Invalid choice");
}

return 0;
}
```

OUTPUT:

```
Number of memory partitions: 5
Number of processes: 4
Enter the memory partitions:
100
500
200
300
600
Enter process sizes:
212
417
112
426

1. First Fit
2. Best Fit
3. Worst Fit
Enter your choice: 1

212 fits in 500
417 fits in 600
112 fits in 288
426 must wait for its process

...Program finished with exit code 0
Press ENTER to exit console.
```